

第 35 章: Exception Objects (异常对象)

原书:《Learning Python》(6th Edition), 作者 Mark Lutz 本章地位: 本书第一部分(异常处理)的"揭秘"章节。前面几章一直含糊地谈论异常, 本章彻底揭开面纱: 异常到底是什么? 答案是——异常是类实例对象(class instance objects)。本章讲清异常类层次树、内置异常、as 子句与异常对象属性(args、str), 以及自定义异常类的全部惯例, 为下一章(异常处理实战)打基础。

开篇引言

英文原文

So far, this book has been somewhat vague about what an exception actually is. This chapter clears up the mystery by disclosing the facts behind exception objects—both built-in and user-defined. As suggested in the preceding chapters, exceptions are identified by class instance objects. This is what is raised and propagated along by exception processing, and the source of the class matched against except clauses in try statements. Although this means you must use object-oriented programming to define new exceptions in your

programs—and introduces a knowledge dependency lamented in the prior chapter's note—basing exceptions on classes and OOP offers a number of benefits. Among them, class-based exceptions support:

- Flexible exception categories (灵活的异常类别) — Exception classes allow code to choose specificity and ease future changes. Adding new exception subclasses, for example, need not require changes in try statements.
- State information and behavior (状态信息与行为) — Exception classes provide a natural place to store context for use in the try handler. Both attributes and methods, for example, are available on the raised instance.
- Reuse by inheritance (通过继承复用) — Exceptions classes can participate in inheritance hierarchies to obtain and customize common behavior. Inherited error displays, for example, can provide a common look and feel. Because of these advantages, class-based exceptions support program evolution and larger systems well. As you'll learn here, all built-in exceptions are identified by classes and are organized into an inheritance tree for the reasons just listed. You can do the same with user-defined exceptions of your own. In fact, the built-in exceptions we'll study here turn out to be integral to new exceptions you define. Because Python largely requires user-defined exceptions to inherit from built-in exception classes that provide useful defaults for printing and state, the task of coding user-defined exceptions also involves understanding the role

中文翻译

到目前为止，本书对异常到底是什么一直有些含糊其辞。本章将揭开这个谜团，披露异常对象背后的事实——无论是内置异常还是用户自定义异常。正如前面几章所暗示的，异常由类实例对象（class instance objects）来标识。被抛出的、在异常处理过程中被逐层传播的，正是这个对象；try 语句中的 except 子句所要匹配的类，也来源于它。虽然这意味着你必须使用面向对象编程（OOP）来定义程序中的新异常——并且引入了上一章注释中哀叹过的知识依赖——但把异常建立在类和 OOP 之上带来了许多好处。其中，基于类的异常支持：灵活的异常类别——异常类允许代码自己选择精度（specificity），并方便未来的修改。例如，添加新的异常子类，不需要修改现有的 try 语句。状态信息与行为——异常类提供了一个天然的位置，用来存放供 try 处理器使用的上下文。例如，在被抛出的实例上，既有属性也有方法可用。通过继承复用——异常类可以参与继承层次结构，从而获得并定制公共行为。例如，继承而来的错误显示可以为所有异常提供统一的外观和风格。正因为这些优点，基于类的异常能很好地支持程序演进和大规模系统。你将在本章学到：所有内置异常都由类标识，并且出于上述原因被组织成一棵继承树。你也可以为自定义异常做同样的事情。事实上，我们本章要研究的内置异常，与你定义的新异常密不可分。因为 Python 在很大程度上要求用户自定义异常继承自内置异常类——这些内置类提供了打印和状态方面有用的默认实现——所以编写用户自定义异常这一任务，本身也涉及理解这些内置类的角色。

深度理解

·核心概念：本章最核心的一句话就是"异常是对象"。raise 抛出的不是字符串、不是数字，而是一个类实例对象；except 子句匹配的不是"错误名字"，而是"类"。这是理解全书异常机制的总开关。

·底层实现：在 CPython 中，异常传播过程就是把一个异常实例（以及它的 traceback 链）在栈帧之间传递。except 子句的匹配本质上是类的 isinstance 检查——except E: 等价于"被抛出的实例是不是 E 或其子类的实例"。

·设计原因：Python 1.5 之前异常可以是字符串，但字符串无法携带状态、无法继承、无法区分"类别"。把异常升级为类，是 Python 语言史上一次著名的"面向对象化"改造（详见章末技术拓展）。作者特别强调的三个好处：灵活类别、状态与行为、继承复用，正是这次改造的动机。

·实际问题：大项目中异常要跨越模块文件传播（raise 和 try 可能在不同文件里），用类做异常，状态才能可靠地随异常一起走；用全局变量传状态会因"不知道 globals 在哪个文件"而不可靠（本章后面会讲）。

·初学者误区：很多人以为 except 子句是"按名字精确匹配"，于是为每一个异常类型写一个 except。实际上 except 是"按继承树匹配"——抓到超类就等于抓到所有子类。这是本章反复强调的重点。

Exception Classes (异常类)

英文原文

Whether built-in or user-defined, exceptions work much of their magic by superclass relationships: a raised exception matches an `except` clause if that clause names the exception's class or any superclass of it. Put another way, a `try` statement's `except` matches both the class it lists, as well as all of that class's subclasses lower in the class tree. The net effect is that class exceptions naturally support the construction of exception hierarchies: superclasses become category names, and subclasses become specific kinds of exceptions within a category. By naming a general exception superclass, an `except` clause can catch an entire category of exceptions—any more specific subclass will match. In addition to this category idea, class-based exceptions support exception state information and allow exceptions to inherit common behaviors, as noted. To see how all these assets come together in code, let's turn to an example.

中文翻译

无论是内置异常还是用户自定义异常，异常的大部分魔力都来自超类关系（`superclass relationships`）：一个被抛出的异常能匹配某个 `except` 子句，当且仅当该子句写的是这个异常类的名字，或者是它的任何一个超类的名字。换句话

说，try 语句的 except 不仅匹配它列出的那个类，还匹配类树中位于其下方的该类的所有子类。最终效果是：类异常天然地支持构造异常层次结构（exception hierarchies）——超类成为"类别"（category）的名字，子类成为该类别中"具体"（specific）的异常种类。通过在 except 子句中写一个通用的异常超类，就能捕获一整类异常——任何更具体的子类都会被匹配。除了这种"类别"思想，基于类的异常还支持异常状态信息，并允许异常继承公共行为，如前所述。要看这些优点如何在代码中结合起来，让我们来看一个例子。

深度理解

- 核心概念：异常匹配是继承关系匹配，不是名字匹配。except General: 能抓到 Specific1 和 Specific2 的实例，因为它们都是 General 的子类。

- 底层实现：解释器在匹配 except 子句时，对被抛出的实例执行 isinstance(instance, ExceptClass) 检查。isinstance 内部就是沿 mro（方法解析顺序，method resolution order）向上查找类型。所以匹配的本质是"沿类树向上走"。

- 设计原因：把"类别"与"具体类型"分开，正是为了让处理代码面向抽象层编写。这跟 OOP 里的"面向接口编程"是同一个道理：客户代码只依赖稳定的超类，不依赖会变化的子类。

- 实际问题：设想一个数据库库抛 DBError，其下有 ConnectionError、QueryError。调用方写一个 except DBError 就能覆盖全部——将来库加新子类 TimeoutError，调

用方代码一个字都不用改。

·初学者误区：误以为 `except` 子句的顺序和 `elif` 一样"匹配第一个满足的"就不继续——对基本 `except` 而言确实如此，但很多新手以为后写的 `except Specific` 在 `except General` 之后仍有机会执行。不会！`try` 只运行一个匹配的处理器。所以子类（具体的）要写在超类（通用的）之前。

Coding Exceptions Classes（编写异常类）

英文原文

In the file listed in Example 35-1, `categoric.py`, we define a superclass called `General` and two subclasses called `Specific1` and `Specific2`. This example illustrates the notion of exception categories—`General` is a category name, and its two subclasses are specific types of exceptions within the category. Handlers that catch `General` will also catch any subclasses of it, including `Specific1` and `Specific2`. Example 35-1. `categoric.py`

```
python class General(Exception): pass class Specific1(General): pass class Specific2(General): pass def raiser0(): X = General() # Raise superclass instance raise X def raiser1(): X = Specific1() # Raise subclass instance raise X def raiser2(): X = Specific2() # Raise different subclass instance raise X for func in (raiser0, raiser1, raiser2): try: func()
```

except General: # Match General or any subclass of it
import sys print('caught:', sys.excinfo()[0]) When this
example runs, its try statement catches and reports in
stances of all three of its classes because the except c
lause names their common superclass:

```
$ python3 categoric.py caught: <class'main.General'  
> caught: <class'main.Specific1'> caught: <class'mai  
n.Specific2'>
```

This code is mostly straightforward, but here are a fe
w points to notice: Exception superclass (异常超类)
— Classes used to build exception category trees hav
e very few requirements—in fact, in this example, the
y are mostly empty, with bodies that do nothing but
pass. Notice, though, how the top-level class here inh
erits from the built-in Exception class. This is require
d: classes that don't inherit from a built-in exception
class won't work in most exception contexts. The buil
t-in superclass is normally Exception, the root for non
exit exceptions, but may also be BaseException, the r
oot for all exceptions, or other. Although we don't e
mploy it here, Exception provides behavior you'll me
et later that makes inheriting from it useful, required
or not. Raising instances (抛出实例) — In this code,
we call classes to make instances for the raise statem
ents (notice the parentheses). In the class exception
model, we always raise and catch a class instance obj
ect. If we list a class name without parentheses in a r
aise, Python makes an instance for us by calling the c

lass with no constructor arguments. Exception instances can be created before the raise, as done here, or within the raise statement itself. Catching categories (捕获类别) — This code includes functions that raise instances of all three of our classes as exceptions, as well as a top-level try that calls the functions and catches General exceptions. The same try also catches the two specific exceptions because they are subclasses of General—that is, members of its category. Exception details (异常细节) — The exception handler here uses the sys.excinfo call, which is one way to fetch the exception being handled in a generic fashion. As you'll see in more detail in the next chapter, the first item in this call's result tuple is the class of the exception raised, and the second is the actual instance raised. In a general except clause like the one here that catches all classes in a category, sys.excinfo can be used to determine exactly what has occurred. The last point merits elaboration. When an exception is caught, we can be sure that the instance raised is an instance of the class listed in the except or one of its subclasses. Because of that, the specific kind of exception raised can also be had via the type result or class attribute of the instance, regardless of how the instance is obtained. To demo, the variant in Example 35-2 works the same as the prior example but uses the as extension in its except clause to directly assign a variable to the instance raised, from which type yields the exception'

s kind. Example 35-2. categoric2.py

```
python class General(Exception): pass class Specific1(
General): pass class Specific2(General): pass def raiser
0(): raise General() def raiser1(): raise Specific1() def r
aiser2(): raise Specific2() for func in (raiser0, raiser1, r
aiser2): try: func()
```

except General as X: # X is the raised instance print('c
aught:', type(X)) # Same as sys.excinfo()[0], X.class Be
cause the except's as can be used to access the exce
ption directly this way, sys.excinfo is more useful for
empty except clauses that do not otherwise have a w
ay to access the instance or its class. More importantl
y, well-designed programs usually should not have to
care about which specific exception was raised at all
—calling methods of the exception instance should a
utomatically dispatch to behavior tailored for the exc
eption raised. There's more on this and sys.excinfo an
d its ilk in the next chapter. Also, see Chapter 29 and
Part VI at large if you've forgotten what class means i
n an instance, and the prior chapter for a review of th
e as used here.

中文翻译

在示例 35-1 的文件 categoric.py 中，我们定义了一个名为 General 的超类和两个名为 Specific1、Specific2 的子类。这个例子说明了“异常类别”的概念——General 是一个类别名，它的两个子类是该类别中具体的异常类型。捕获 General 的处理器也会捕获它的任何子类，包括 Specific1

和 Specific2。示例 35-1. categoric.py python class General(Exception): pass class Specific1(General): pass class Specific2(General): pass def raiser0(): X = General() # 抛出超类实例 raise X def raiser1(): X = Specific1() # 抛出一个子类实例 raise X def raiser2(): X = Specific2() # 抛出一个不同的子类实例 raise X for func in (raiser0, raiser1, raiser2): try: func() except General: # 匹配 General 或其任何子类 import sys print('caught:', sys.excinfo()[0])

运行这个例子时，它的 try 语句捕获并报告了所有三个类的实例，因为 except 子句写的是它们的公共超类：\$ python3 categoric.py caught: <class'main.General'> caught: <class'main.Specific1'> caught: <class'main.Specific2'> 这段代码大部分很直白，但有几个要点值得注意：

异常超类——用于构建异常类别树的类几乎没什么要求——事实上，在这个例子中它们基本是空的，类体里只有 pass。不过请注意，这里的顶层类继承自内置的 Exception 类。这是必须的：不继承自内置异常类的类，在大多数异常上下文中都无法工作。内置超类通常是 Exception——非退出类异常（nonexit exceptions）的根类——但也可以是 BaseException——所有异常的根类——或其他类。虽然这里没有用到，Exception 提供的那些行为（你稍后会遇到）使得继承它是有用的，无论是否必须。抛出实例——在这段代码中，我们调用类来为 raise 语句创建实例（注意括号）。在类异常模型中，我们总是抛出并捕获一个类实例对象。如果在 raise 中只写类名而不带括号，Python 会通过“以无构造参数调用该类”的方式替我们创建一个实例。异常实例可以在 raise 之前创建（像这里这样），也可以在 raise 语句本身中创建。捕获类别——这段代码包含了把三个类

的实例作为异常抛出的函数，以及一个调用这些函数并捕获 General 异常的顶层 try。同一个 try 也捕获了那两个具体异常，因为它们是 General 的子类——也就是说，它们是该类别的成员。异常细节——这里的异常处理器使用了 sys.excinfo 调用，这是以通用方式获取当前正在处理的异常的方法之一。你会在下一章看到更多细节：该调用返回的元组的第一个元素是被抛出异常的类，第二个元素是被抛出的实际实例。在像这里这样“捕获一个类别中所有类”的通用 except 子句中，可以用 sys.excinfo 来确定到底发生了什么。最后一点值得展开说明。当一个异常被捕获时，我们可以确定：被抛出的实例是 except 中所列类（或其子类）的实例。正因如此，无论实例是如何获得的，被抛出的异常的具体种类都可以通过 type 的结果或实例的 class 属性得知。为了演示，示例 35-2 中的变体与前一示例运行结果相同，但在其 except 子句中使用了 as 扩展，直接把一个变量赋给了被抛出的实例，用 type 就能得到异常的种类。示例 35-2. categoric2.py

```
python class General(Exception): pass
class Specific1(General): pass
class Specific2(General): pass
def raiser0(): raise General()
def raiser1(): raise Specific1()
def raiser2(): raise Specific2()
for func in (raiser0, raiser1, raiser2):
    try: func()
    except General as X:
        # X 就是被抛出的实例
        print('caught:', type(X))
# 等价于 sys.excinfo()[0] 或 X.class
因为 except 的 as 可以这样直接访问异常，sys.excinfo 对于那些空 except 子句（没有任何其他方式访问实例或其类的子句）更有用。更重要的是，设计良好的程序通常根本不该关心具体抛出了哪个异常——调用异常实例的方法应该自动分派（dispatch）到为被抛出异常量身定做的行为上。关于这些内容以及 sys.excinfo
```

o 之类工具的更多细节在下一章。另外，如果你忘了实例中 class 的含义，请看第 29 章和第 VI 部分；如果想复习这里用到的 as，请看上一章。

代码分析

```
class General(Exception): pass           # Exce...
class Specific1(General): pass           # 1 General
class Specific2(General): pass           # 2 General

def raiser0():
    X = General()                         #
    raise X                               #
def raiser1():
    X = Specific1()                       #
    raise X
def raiser2():
    X = Specific2()                       #
    raise X
for func in (raiser0, raiser1, raiser2):
    try:
        func()                           #
    except General:                       # General
        import sys
        print('caught:', sys.exc_info()[0]) # ""
```

解释器如何处理：循环中三次调用 func()，每次抛出的分别是 General、Specific1、Specific2 的实例。解释器在匹配 except General 时对实例做 isinstance 检查：General 实例当然匹配；Specific1、Specific2 实例因为 mro 中都有 General，同样匹配。也就是说，一个 except 覆盖了整棵子树。sys.exc_info() 返回 (异常类, 异常实例, traceback 对象) 三元组，[0] 取出的就是类的对象本身，所以打印出 <class'main.Specific1'> 这样的形式。注意 im

port sys 写在 except 块里——except 块就是普通代码块，想在哪导入就在哪导入。

- except General as X: as X 把被抛出的实例绑定到变量 X。type(X) 与 X.class、sys.excinfo()[0] 三种写法指向同一个东西——实例的类。这是"异常是对象"的直接体现：实例在手，类、属性、方法全都有。

- 设计思想：抛出端可以抛出任何子类，捕获端只需要面向超类编程。这实现了松耦合——抛异常的代码自由演进，捕获代码保持稳定。作者特别提醒：写 raise General（不带括号）也可以，Python 会替你 General(); 但显式加括号、显式创建实例是更清晰的表达。

深度理解

- 核心概念：自定义异常三步走：① 继承 Exception；② 用 pass 或自定义内容填充类体；③ raise 时创建并抛出实例。

- 底层实现：类体为空的类依然有构造函数——从 BaseException.init 继承的默认构造函数，它会把所有参数存进 args 元组属性。raise General 与 raise General() 的区别只是：前者由解释器补一个"无参调用"，后者由程序员显式调用，结果完全一样。

- 设计原因：为什么要"异常是实例"而不是"异常是类"？因为实例才有自己的状态——每个被抛出的异常记录各自的错误信息，args、自定义属性都挂在实例上。类只是"模板"，实例才是"事件"。

·实际问题: `sys.excinfo()` 与 `as` 的区别: `as` 需要 `except` 子句指名类; 而 `except:` (裸 `except`) 没有指名任何类, 此时只能用 `sys.excinfo()` 访问异常。不过实践中更推荐 `except Exception as X + as`。

·初学者误区: ① 忘记 (`Exception`) 基类, 写 `class MyError:`——这样的类抛出去后 `except Exception` 抓不到它, 打印也不对; ② 在 `raise` 里写 `raise X()` 而 `X` 已经是实例 (`raise X()` 会尝试把实例当类调用, 报'`X`' object is not callable); ③ 误以为 `as X` 绑定的是类, 其实绑定的是实例。

Why Exception Hierarchies? (为什么需要异常层次结构?)

英文原文

Because there are only three possible exceptions in the prior section's examples, it doesn't really do justice to the utility of class exceptions. In principle, we could achieve the same effects by coding a list of exception names in a parenthesized tuple within the `except` clause:

```
python try: func()
except (General, Specific1, Specific2): # Catch any of these...
```

This approach may work for smaller, self-contained c

ode. For large or high exception hierarchies, however, it will probably be easier to catch categories using class-based categories than to list every member of a category in a single except clause. Perhaps more importantly, you can extend exception hierarchies as software needs evolve by adding new subclasses without breaking existing handler code.

Suppose, for example, you code a numeric programming library in Python to be used by a large number of people. While you are writing your library, you identify two things that can go wrong with numbers in your code—division by zero and numeric overflow. You document these as the two standalone exceptions that your library may raise:

```
python # mathlib.py class Divzero(Exception): pass
class Oflow(Exception): pass
def func():...
raise Divzero()... and so on ...
```

Now, when people use your library, they typically wrap calls to your functions or classes in try statements that catch your two exceptions; after all, if they do not catch your exceptions, exceptions from your library will kill their code:

```
python # client.py import mathlib
try: mathlib.func()
except (mathlib.Divzero, mathlib.Oflow):... handle and
recover ...
```


This works fine, and lots of people start using your library. Six months down the road, though, you revise it (as programmers are prone to do). Along the way, you identify a new thing that can go wrong—underflow, perhaps—and add that as a new exception:

```
python # mathlib.py class Divzero(Exception): pass  
class Oflow(Exception): pass  
class Uflow(Exception): pass  
Unfortunately, when you re-release your code, you create a maintenance problem for your users. If they've listed your exceptions explicitly, they now have to go back and change every place they call your library to include the newly added exception name:
```

```
python # client.py try: mathlib.func()  
except (mathlib.Divzero, mathlib.Oflow, mathlib.Uflow):... handle and recover ...
```

This may not be the end of the world. If your library is used only in-house, you can make the changes yourself. You might also ship a Python script that tries to fix such code automatically (it would probably be only a few dozen lines, and it would guess right at least some of the time). If many people have to change all their try statements each time you alter your exception set, though, this is not exactly the politest of upgrade policies.

Your users might try to avoid this pitfall by coding empty except clauses to catch all possible exceptions:

```
python # client.py try: mathlib.func()
except: # Catch everything here (or catch Exception)
... handle and recover ...
```

But as noted in the prior chapter, this workaround might catch more than they bargained for—things like running out of memory, keyboard interrupts (Ctrl+C), system exits, and even typos in their own try block's code will all trigger exceptions, and such things should pass, not be caught and erroneously classified as library errors. Catching the Exception superclass improves on this but still intercepts—and thus may mask—program errors.

And really, in this scenario, users want to catch and recover from only the specific exceptions the library is defined and documented to raise. If any other exception occurs during a library call, it's likely a genuine bug in the library (and it's probably time to contact the vendor). As a rule of thumb, it's usually better to be specific than general in exception handlers—an idea we'll revisit as a "gotcha" in the next chapter.

So what to do, then? In principle again, the library module could provide a tuple object that contains all the exceptions it can possibly raise. The client could then import the tuple and name it in an except clause to catch all the library's exceptions (recall that using a tuple catches any of its exceptions). This would work and support mods, but you'd need to keep the tuple

e up-to-date with library exceptions, and that's both error-prone and tedious.

Class exception hierarchies solve this dilemma better. Rather than defining your library's exceptions as a set of autonomous classes, arrange them into a class tree with a common superclass to encompass the entire category:

```
python # mathlib.py class NumErr(Exception): pass  
class Divzero(NumErr): pass  
class Oflow(NumErr): pass  
def func():...
```

```
raise DivZero()... and so on ...
```

This way, users of your library simply need to list the common superclass (i.e., category) to catch all of your library's exceptions—both now and in the future:

```
python # client.py import mathlib  
try: mathlib.func()  
except mathlib.NumErr:... handle and recover ...
```

When you go back and hack (update) your code again now, you can add new exceptions as new subclasses of the common superclass:

```
python # mathlib.py...
```

```
class Uflow(NumErr): pass
```

The end result is that user code that catches your library's exceptions will keep working, unchanged. In fact, you are free to add, delete, and change exceptions arbitrarily in the future—as long as clients name the superclass, and that superc

lass remains intact, they are insulated from changes in your exceptions set.

In other words, class exceptions provide a better answer to maintenance issues than other solutions can.

Class-based exception hierarchies also support state retention and inheritance in ways that make them ideal in larger programs. To understand these roles, though, we first need to see how user-defined exception classes relate to the built-in exceptions from which they inherit.

中文翻译

因为前一节示例里只有三种可能的异常，并不能真正体现类异常的用处。原则上，我们也可以在 `except` 子句里用一个带括号的元组列出异常名，达到同样的效果：`python try: func() except (General, Specific1, Specific2): # 捕获其中任意一种...`这种方法对较小、自包含的代码或许可行。但对大型的、层次分明的异常体系来说，用基于类的“类别”来捕获，很可能比在单个 `except` 子句里列出类别的每个成员更容易。或许更重要的是：随着软件需求演进，你可以通过添加新子类来扩展异常层次结构，而不破坏现有的处理器代码。例如，假设你用 Python 编写了一个供大量人群使用的数值计算库。在编写库的过程中，你发现代码中数字可能出现两种问题——除以零和数值溢出。你把它们作为库可能抛出的两个独立异常记录下来：`python # mathlib.py class Divzero(Exception): pass class Oflow(Exception): pass def func():...raise Divzero()...`等等...现在，当人们使用你的库时，他们通常会把对库函数或类的调用包在捕获这两个

异常的 try 语句里；毕竟，如果不捕获你的异常，库抛出的异常就会干掉他们的代码：python # client.py import mathlib try: mathlib.func() except (mathlib.Divzero, mathlib.Oflow):...处理并恢复...这样工作得很好，很多人开始使用你的库。但六个月后，你（程序员改代码是常态）修订了它。在修订过程中，你发现了新的出错点——也许是下溢（underflow）——于是把它加为新的异常：python # mathlib.py class Divzero(Exception): pass class Oflow(Exception): pass class Uflow(Exception): pass 不幸的是，当你重新发布代码时，你给用户制造了一个维护问题。如果他们明确列出了你的异常，现在就得回去修改每一处调用你库的地方，把新加的异常名补进去：python # client.py try: mathlib.func() except (mathlib.Divzero, mathlib.Oflow, mathlib.Uflow):...处理并恢复...这也许不是世界末日。如果你的库只在公司内部使用，你可以自己改。你也可以发布一个试图自动修复此类代码的 Python 脚本（大概只需几十行，而且至少有时候能猜对）。但如果每次你改动异常集合，都有许多人必须修改他们所有的 try 语句，这可不是什么礼貌的升级策略。你的用户可能会试图用空 except 子句来规避这个坑，捕获所有可能的异常：python # client.py try: mathlib.func() except: # 在这里捕获一切（或捕获 Exception）...处理并恢复...但正如上一章所述，这种变通方案捕获的可能超出了他们想要的——像内存耗尽、键盘中断（Ctrl+C）、系统退出，甚至他们自己 try 块代码里的拼写错误，都会触发异常，而这些东西应该放行，不该被捕获并被错误地归类为库错误。捕获 Exception 超类有所改进，但仍然会拦截——从而可能掩盖——程序错误。事实上，在这种场景下，用户只想捕获并恢复那些库定义并文

档化了要抛出的特定异常。如果库调用期间发生了任何其他异常，那很可能是库本身的真 bug（也许该联系厂商了）。作为经验法则，异常处理器里具体优于通用——这个观点我们会在下一章作为“陷阱”（gotcha）再次讨论。那么该怎么办呢？再说一次原则：库模块可以提供一个包含它所有可能异常对象的元组。客户端导入这个元组并在 except 子句中引用它，就能捕获库的所有异常（回忆一下：使用元组会捕获其中任意一种异常）。这能工作并支持修改，但你需要让元组与库的异常保持同步，这既容易出错又繁琐。类异常层次结构能更好地解决这个两难困境。与其把库的异常定义为一组彼此独立的类，不如把它们组织成一棵以公共超类为根、涵盖整个类别的类树：python # mathlib.py class NumErr(Exception): pass class Divzero(NumErr): pass class Oflow(NumErr): pass def func():...raise DivZero()...等等...这样一来，库的用户只需列出公共超类（即“类别”），就能捕获库的所有异常——无论现在还是将来：python # client.py import mathlib try: mathlib.func() except mathlib.NumErr:...处理并恢复...现在你回去继续改（更新）代码时，可以以公共超类新子类的形式添加新异常：python # mathlib.py...class Uflow(NumErr): pass 最终结果是：捕获你库异常的用户代码会继续工作，无需修改。事实上，将来你可以随意添加、删除和更改异常——只要客户端引用的是超类、且超类保持完整，他们就与异常集合的变化完全隔离。换句话说，面对维护问题，类异常比任何其他方案都给出了更好的答案。基于类的异常层次结构还以种种方式支持状态保留和继承，这使得它们在更大的程序中近乎理想。但要理解这些角色，我们首先需要看看用户自定义异常类与它们所继承的内置异常之间是什么关系。

深度理解

·核心概念：异常层次结构的价值在于开闭原则——对扩展开放（加子类），对修改关闭（客户代码不动）。这是本书用整整一小节讲一个 `mathlib.py` 例子的原因。

·底层实现：匹配是 `isinstance` 检查，沿 `mro` 走。`except (Divzero, Oflow)` 的元组形式在解释器内部逐个做 `isinstance` 检查，本质是“精确列举”；而 `except NumErr` 是“按树匹配”，一条路径覆盖整棵树。

·设计原因：三个方案对比——①元组列举：同步维护麻烦；②裸 `except/except Exception`：误伤过多，掩盖 bug；③类层次：超类成为稳定的“契约”，这正是把 `NumErr` 当“类别”的用意。作者的经验法则：“具体优于通用”，但“类别优于列举”。

·实际问题：库设计是真实工程问题——标准库（如 `OSError` 体系）和第三方库（如 `requests` 的 `RequestException`）都用这个模式。你写库时给用户一个超类，用户就能安心 `except` 你的类别。

·初学者误区：①把裸 `except:` 当“万能安全网”——它连 `KeyboardInterrupt`、`SystemExit` 都吞，而且掩盖拼写错误等真 bug；②以为 `except Exception` 万能——它放过了 `SystemExit/KeyboardInterrupt/GeneratorExit`，这是设计使然（下面小节会讲）；③在库代码里定义异常但不继承 `Exception`，用户想 `except Exception` 统一兜底时抓不到。

Built-in Exception Classes (内置异常类)

英文原文

The prior section's example wasn't really pulled out of thin air. All built-in exceptions that Python itself may raise are predefined class objects. Moreover, they are organized into a shallow hierarchy with general superclass categories and specific subclass types, much like the prior section's final exceptions class tree. All the familiar exceptions you've seen (e.g., `SyntaxError`) are really just predefined classes, available as built-in names in the module named `builtins`. In addition, Python organizes the built-in exceptions into a hierarchy to support a variety of catching modes. For example:

- `BaseException`: topmost root, with printing and constructor defaults — The top-level root superclass of exceptions. This class is not supposed to be directly inherited by user-defined classes (use `Exception` instead).
- It provides default printing and state retention behavior inherited by subclasses. If the `str` built-in is called on an instance of this class (e.g., by `print`), the class returns the display strings of the constructor arguments passed when the instance was created (or an empty string if there were no arguments). In addition, unless subclasses replace this class's constructor, all of the arguments passed to this class at instance construction time are stored in its `args` attribute as a tuple. E

exception: root of user-defined exceptions — The top-level root superclass of application-related exceptions. This is an immediate subclass of `BaseException` and is a superclass to every other built-in exception, except the system exit event classes (`SystemExit`, `KeyboardInterrupt`, and `GeneratorExit`) and an exception-group class we'll ignore here. Nearly all user-defined classes should inherit from this class, not `BaseException`. When this convention is followed, naming `Exception` in a try statement's handler ensures that your program will catch everything but system exit events, which should normally be allowed to pass. In effect, `Exception` becomes a catchall in try statements but is more accurate than an empty `except`.

`ArithmeticError`: root of numeric errors — A subclass of `Exception`, and the superclass of all numeric errors. Its subclasses identify specific numeric errors: `OverflowError`, `ZeroDivisionError`, and `FloatingPointError`.

`LookupError`: root of indexing errors — A subclass of `Exception`, and the superclass category for indexing errors for both sequences and mappings: `IndexError` and `KeyError`.

`OSError`: root of IO and other system-function errors, with details — A subclass of `Exception`, with attributes for error details (e.g., `errno`, `strerror`, and `filename`), and subclasses for specific errors: `FileNotFoundError`, `PermissionError`, `TimeoutError`, and more. And so on—because the built-in exception set is prone to frequent changes, this book doesn't document it exhaustively. You can

n read further about its contents and structure in the Python library manual.

中文翻译

上一节的例子并非凭空捏造。Python 自身可能抛出的所有内置异常，都是预定义的类对象。而且，它们被组织成一棵浅层层次结构——通用的超类类别加上具体的子类类型，与上一节最终的异常类树非常相似。你见过的所有熟悉的异常（如 `SyntaxError`）其实都只是预定义的类，作为内置名字存在于名为 `builtins` 的模块中。此外，Python 把内置异常组织成一个层次结构，以支持各种捕获模式。例如：`BaseException`：最顶层的根，带打印和构造默认行为——异常的顶级根超类。这个类不应该被用户自定义类直接继承（要用 `Exception` 代替）。它提供了子类继承的默认打印和状态保留行为。如果对这个类的实例调用 `str` 内置函数（比如通过 `print`），该类会返回创建实例时传入的构造参数的显示字符串（如果没有参数则返回空字符串）。此外，除非子类替换了这个类的构造函数，否则在实例构造时传给该类的所有参数，都会以元组形式存入它的 `args` 属性。`Exception`：用户自定义异常的根——与应用程序相关异常的顶级根超类。它是 `BaseException` 的直接子类，并且是除系统退出事件类（`SystemExit`、`KeyboardInterrupt` 和 `GeneratorExit`）以及一个这里忽略的异常组类（`exception-group class`）之外、所有其他内置异常的超类。几乎所有用户自定义类都应该继承这个类，而不是 `BaseException`。遵循这一惯例时，在 `try` 语句的处理器中写 `Exception`，就能确保你的程序捕获除系统退出事件之外的一切——而系统退出事件通常应当放行。实际上，`Exception` 在 `try` 语句中

成了"万能捕获", 但它比空 `except` 更准确。 `ArithmeticError`: 数值错误的根——`Exception` 的子类, 是所有数值错误的超类。它的子类标识具体的数值错误: `OverflowError`、`ZeroDivisionError` 和 `FloatingPointError`。 `LookupError`: 索引错误的根——`Exception` 的子类, 是序列和映射索引错误的超类类别: `IndexError` 和 `KeyError`。 `OSError`: IO 及其他系统函数错误的根, 带详细信息——`Exception` 的子类, 带有错误详情的属性 (如 `errno`、`strerror` 和 `filename`) , 以及具体错误的子类: `FileNotFoundError`、`PermissionError`、`TimeoutError` 等等。如此等等——因为内置异常集合变动频繁, 本书不打算穷尽记录它。你可以查阅 Python 库手册 (library manual) 了解它的更多内容和结构。

深度理解

·核心概念: 内置异常是一棵浅层类树: `BaseException` (根) → `Exception` → 各类别超类 (`ArithmeticError`、`LookupError`、`OSError` 等) → 具体异常 (`ZeroDivisionError`、`IndexError`、`FileNotFoundError` 等) 。 `builtins` 模块自动在全局命名空间提供这些名字。

·底层实现: `BaseException` 是唯一真正的根。它实现了默认的 `init` (把参数存进 `args` 元组) 和 `str/repr` (显示参数) 。 `SystemExit`、`KeyboardInterrupt`、`GeneratorExit` 直接继承 `BaseException` 而不是 `Exception`——这正是 `except Exception` 抓不到 `Ctrl+C` 的原因。在 CPython 里, `traceback` 链是挂在异常实例上的 `traceback` 属性。

·设计原因：为什么要有 `BaseException` 和 `Exception` 两层根？为了把"程序正常错误"与"解释器级事件"分开：`KeyboardInterrupt`（用户按 Ctrl+C）、`SystemExit`（`sys.exit()` 调用）是"请退出"的信号，一般不应被业务代码吞掉。`Exception` 是"应用程序级错误"的边界，是 `try` 语句的默认安全网。

·实际问题：这套分类直接指导实践——想处理"所有的普通错误"用 `except Exception`；想处理"数字错误"用 `except ArithmeticError`（覆盖 `ZeroDivisionError`、`OverflowError`、`FloatingPointError`）；想处理"索引错误"用 `except LookupError`（覆盖 `IndexError`、`KeyError`）。OS Error 家族在文件 IO 时代意义重大，其 `errno/filename` 属性给出了出错细节。

·初学者误区：① 直接继承 `BaseException` 自定义异常——文档明确不建议，会破坏 `except Exception` 的兜底约定；② 以为 `except Exception` 什么都能抓——它抓不到 `SystemExit` 等三个"退出事件"类；③ 以为内置异常名字是一堆"魔法符号"——它们就是普通的类，可以 `type(ZeroDivisionError)` 验证，也可以 `isinstance(e, ArithmeticError)` 判断。

Built-in Exception Categories（内置异常类别）

英文原文

The built-in class tree allows you to choose how specific or general your handlers will be. For example, because the built-in exception `ArithmeticError` is a superclass for more specific exceptions such as `OverflowError` and `ZeroDivisionError`:

- By listing `ArithmeticError` in a `try`, you will catch any kind of numeric error raised.
- By listing `ZeroDivisionError`, you will intercept just that specific type of error and no others.

Similarly, because `Exception` is the superclass of all application-level exceptions, you can generally use it as a catchall—as outlined in the prior chapter, the effect is much like an empty `except`, but it allows system exit exceptions to pass and propagate as they usually should:

```
python try:...  
  
except Exception: # Exits not caught here... handle all  
application exceptions ...  
  
else:... handle no-exception case ...
```

This technique is reliable because Python requires all classes to derive from built-in exceptions. Still, this scheme suffers most of the same potential pitfalls as the empty `except`, as described in the prior chapter—it might intercept exceptions intended for elsewhere, and it might mask genuine programming errors. Since this is such a common issue, we'll revisit it one more

time as a "gotcha" in the next chapter.

Whether or not you will leverage the categories in the built-in class tree, it serves as a good example. By using similar techniques for class exceptions in your own code, you can provide exception sets that are flexible and easily modified.

中文翻译

内置类树允许你选择处理器要多具体或多通用。例如，由于内置异常 `ArithmeticError` 是 `OverflowError`、`ZeroDivisionError` 等更具体异常的超类：- 在 `try` 中列出 `ArithmeticError`，你将捕获抛出的任何数值错误。- 列出 `ZeroDivisionError`，你将只拦截那一种具体错误，不碰其他。类似地，因为 `Exception` 是所有应用程序级异常的超类，你通常可以把它用作“万能捕获”——如上一章所述，效果与空 `except` 很相似，但它允许系统退出异常照常放行并传播：`python try:...except Exception: # 退出类异常在这里不会被捕获...处理所有应用程序异常...else:...处理无异常的情况...这种技术是可靠的，因为 Python 要求所有类都派生自内置异常。尽管如此，这种方案仍会遭遇与空 except 大部分相同的潜在陷阱，如上一章所述——它可能拦截本应去往别处的异常，也可能掩盖真正的编程错误。因为这是个如此常见的问题，我们会在下一章再把它作为“陷阱”讨论一次。无论你是否利用内置类树中的类别，它都是一个很好的范例。在自己的代码里对类异常采用类似技术，你就能提供灵活、易修改的异常集合。`

代码分析

```
try:
    ...
except Exception:
    ...
else:
    ...
```


SystemExit ...

#

·解释器如何处理：try 先执行主体；若抛出异常，解释器自上而下匹配 except 子句，对实例做 isinstance 检查。except Exception 匹配所有 Exception 子类的实例；SystemExit、KeyboardInterrupt、GeneratorExit 因为不在 Exception 的子树里，直接穿透，继续向外传播。若主体没抛异常，则执行 else 块。

·设计思想：except Exception 是一个“精确的万能捕获”——它在“什么都不漏”和“什么都别乱抓”之间取平衡。这也是为什么下一章会把它作为最重要的 gotcha 之一反复讨论。

深度理解

·核心概念：内置类树 = 一套“捕获粒度”选择器。同一棵树上，ArithmeticError（粗粒度）、ZeroDivisionError（细粒度）各取所需。粒度选择决定异常处理器是“分类处理”还是“统一兜底”。

·底层实现：所有类都必须派生自内置异常，这是 CPython 在抛异常时做的检查——raise 一个不是 BaseException 实例的东西会报 TypeError: exceptions must derive from BaseException。正是这条规则保证了 except Exce

ption 的兜底可靠性。

·设计原因：空 `except:` 之所以被嫌弃，是因为它过于贪婪——连 `KeyboardInterrupt`、`SystemExit` 和程序 bug 都吞。`except Exception` 是 Python 给的"体面版本"：普通错误全抓，退出事件放行。

·实际问题：主程序顶层经常写 `except Exception + 日志记录 + 兜底处理`，防止任何未预期错误把整个服务干崩；同时保留 `Ctrl+C` 退出能力。

·初学者误区：① 分不清 `except:` 与 `except Exception:`——前者还会捕获 `BaseException` 的三大退出类，后者不会；② 以为"捕获粒度越粗越安全"——实际上粗捕获会掩盖 bug（比如把 `AttributeError` 当成库错误处理），调试时异常在 `except` 里被吞掉、程序带着错误状态继续跑，比崩溃更难查。

Default Printing and State（默认打印与状态）

英文原文

Built-in exceptions also provide default print displays and state retention, which is often as much logic as user-defined classes require. Unless you redefine the constructors your classes inherit from built-ins, any constructor arguments you pass to these classes are automatically saved in the instance's `args` tuple attribute and are automatically displayed when the instance i

s printed.

An empty tuple and display string are used if no constructor arguments are passed, and a single argument displays as itself (not as a tuple) and serves as message details.

This explains why arguments passed to built-in exception classes show up in error messages—any constructor arguments are attached to the instance and displayed when the instance is printed:

```
python >> raise IndexError # Same as IndexError(): no arguments
Traceback (most recent call last): File "<stdin>", line 1, in <module>
IndexError >> raise IndexError('bad') # Constructor argument attached,
printed
Traceback (most recent call last): File "<stdin>", line 1, in <module>
IndexError: bad >> i = IndexError('bad','stuff') # Available in object
attribute "args"
>> i.args ('bad','stuff') >> print(i) # Displays args when printed
manually ('bad','stuff') >> i # Uses repr for echo, str for print
IndexError('bad','stuff')
```

The same holds true for user-defined exceptions because they inherit the constructor and display methods present in their built-in superclasses:

```
python >> class E(Exception): pass >> raise E
Traceback (most recent call last): File "<stdin>", line 1, in <module>
E >> raise E('bad')
Traceback (most recent call last): File "<stdin>", line 1, in <module>
E: bad >> i = E('bad','stuff') >> i.args ('bad','stuff') >> print(i) ('
```

```
bad','stuff') >> i E('bad','stuff')
```

When intercepted in a try statement, the exception instance object gives access to both the original constructor arguments and the display method:

```
python >> try:
```

```
... raise E('bad') # Displays + saves constructor args ...
except E as X: ... print(f'{X} - {X.args} - {X!r}') ... bad - ('
bad',) - E('bad') >> try:
```

```
... raise E('bad','stuff') # Multiple args save/display a tuple ...
except E as X: ... print(f'{X} - {X.args} - {X!r}') ...
('bad','stuff') - ('bad','stuff') - E('bad','stuff')
```

Note that exception instance objects are not strings themselves, but use the str and repr operator-overloading methods we studied in Chapter 30 to provide display strings for print and other contexts. To concatenate with real strings, perform manual conversions: str(X), '%s' % X, f'{X}', and the like.

Although this automatic state and display support is useful by itself, for more specific display and state retention needs, you can always redefine inherited methods such as str and init in Exception subclasses—as the next section shows.

中文翻译

内置异常还提供了默认的打印显示和状态保留，这往往就是用户自定义类所需的全部逻辑。除非你重新定义类从内置类

继承的构造函数，否则传给这些类的任何构造参数都会自动保存在实例的 `args` 元组属性中，并在实例被打印时自动显示。如果没有传构造参数，就使用空元组和空显示字符串；单个参数会以其本身显示（而不是元组形式），并作为消息细节。这解释了为什么传给内置异常类的参数会出现在错误信息中——任何构造参数都被挂到实例上，并在实例被打印时显示出来：

```
python >>> raise IndexError # 等价于 IndexError():
Traceback (most recent call last):
  File "<stdin>", line 1, in <module> IndexError
>>> raise IndexError('bad') # 构造参数被挂到实例上，并被打印出来
Traceback (most recent call last):
  File "<stdin>", line 1, in <module> IndexError: bad
>>> i = IndexError('bad','stuff') # 可以通过对象属性 "args" 访问
>>> i.args ('bad','stuff')
>>> print(i) # 手动打印时显示 args ('bad','stuff')
>>> i # 交互回显用 repr, print 用 str
IndexError('bad','stuff')
用户自定义异常也一样，因为它们继承了内置超类中的构造函数和显示方法：
python >>> class E(Exception): pass
>>> raise E
Traceback (most recent call last):
  File "<stdin>", line 1, in <module> E
>>> raise E('bad')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module> E: bad
>>> i = E('bad','stuff')
>>> i.args ('bad','stuff')
>>> print(i) ('bad','stuff')
>>> i
E('bad','stuff')
当在 try 语句中拦截到时，异常实例对象既能访问原始的构造参数，也能访问显示方法：
python >>> try: ... raise E('bad') # 既保存构造参数又显示...
except E as X: ... print(f'{X} - {X.args} - {X!r}') ... bad - ('bad',) - E('bad')
>>> try: ... raise E('bad','stuff') # 多个参数保存/显示为一个元组...
except E as X: ... print(f'{X} -
```

{X.args} - {X!r}') ... ('bad','stuff') - ('bad','stuff') - E('bad','stuff') 注意：异常实例对象本身并不是字符串，而是利用我们在第 30 章学过的 str 和 repr 运算符重载方法，为 print 和其他上下文提供显示字符串。要与真正的字符串拼接，需要手动转换：str(X)、'%s' % X、f'{X}' 等。虽然这种自动的状态和显示支持本身很有用，但若需要更具体的显示和状态保留方式，你随时可以在 Exception 子类中重新定义继承的方法（如 str 和 init）——下一节将展示这一点。

代码分析

```
>>> i = IndexError('bad', 'stuff')    # args
>>> i.args                             # args
('bad', 'stuff')
>>> print(i)                          # print __str__
('bad', 'stuff')
>>> i                                 # __repr__
IndexError('bad', 'stuff')

>>> class E(Exception): pass          #
>>> raise E('bad')                    # =
E: bad                                # args
```

解释器如何处理：IndexError('bad','stuff') 执行构造时，调用从 BaseException 继承的 init，把 ('bad','stuff') 存进实例的 args 属性。print(i) 调用 str(i)，即 str——默认实现返回 str(args)，所以打印出 ('bad','stuff')。交互式回显调用 repr(i)，即 repr——默认实现返回类似构造表达式的文本 IndexError('bad','stuff')。未捕获时，traceback 的最后一行显示规则是：异常类名: 消息；单参数时消息就是该参数（E: bad），多参数时是元组。f-string 里 {X} 用 str，{X!r} 用 repr，所以同一行输出 bad - (

'bad'),) - E('bad') 三种形态。

·设计思想：异常的"零配置可用"正是来自这些默认实现——你自定义异常时什么方法都不用写，传参、存 args、打印全自动。这也解释了为什么"异常参数会出现在错误信息里"。

深度理解

·核心概念：args 元组是异常的"默认状态"，str/repr 是异常的"默认显示"。两者都来自 BaseException 的默认实现，用户异常继承即得。

·底层实现：BaseException.init(args) 只做一件事：self.args = args。BaseException.str 返回 str(args)；当 args 只有一个元素时显示它本身（这解释了 IndexError: bad 而非 IndexError: ('bad',))。traceback 打印异常时也是拿这个字符串拼在类名后面。

·设计原因：Python 想要"简单用例零配置"——raise ValueError("bad value") 直接就有可读的错误信息，不必让每个异常类都写 init/str。这也契合"默认应该合理、特殊才去覆盖"的 Python 哲学。

·实际问题：异常对象不是字符串——新手直接'Error:' + e 会 TypeError，必须先 str(e)。args 属性的存在意味着你能在处理器里拿回原始参数做结构化处理（比如日志、JSON 序列化）。

·初学者误区：① 以为 print(exception) 和 str(exception) 返回一样的东西——print 走 str，交互回显走 repr，两者默认实现不同；② 以为 args 只能有一个元素——它

是元组，可以多个；③ 想在异常类里用 `self.args = ...` 之外的方式存状态，需要重写 `init`（下一节）。

Custom Print Displays (自定义打印显示)

英文原文

As we saw in the preceding section, by default, instances of class-based exceptions display whatever you passed to the class constructor when they are caught and printed:

```
python >> class MyBad(Exception): pass >> try:
... raise MyBad('Sorry--my mistake!') ... except MyBad
as X: ... print(X) ... Sorry--my mistake!
```

This inherited default display model is also used if the exception is displayed as part of an error message when the exception is not caught:

```
python >> raise MyBad('Sorry--my mistake!') Traceback
(most recent call last): File "<stdin>", line 1, in <
module> MyBad: Sorry--my mistake!
```

For many roles, this is sufficient. To provide a more custom display, though, you can define one of two string-representation overloading methods in your class (`repr` or `str`) to return the string you want to display for your exception. The string the method returns will be displayed if the exception either is caught and printed or reaches the default handler:

```
python >> class MyBad(Exception):
def str(self): return'Stuff happens... '>> try:
... raise MyBad() ... except MyBad as X: ... print(X) ... St
uff happens...
>> raise MyBad() Traceback (most recent call last): Fil
e "<stdin>", line 1, in <module> MyBad: Stuff happe
ns...
```

Whatever your method returns is included in error messages for uncaught exceptions and used when exceptions are printed explicitly. The method returns a hardcoded string here to illustrate, but it can also perform arbitrary text processing, possibly using state information attached to the instance object. The next section looks at state information options.

First, though, one fine point: you generally must redefine `str` for exception display purposes because the built-in exception superclasses already have a `str` method, and `str` is preferred to `repr` in some contexts—including error-message displays. If you define a `repr`, printing will happily call the built-in superclass's `str` instead:

```
python >> class Oops(Exception):
def repr(self): return'Custom display not used'>> raise
Oops("Nobody's perfect") ... Oops: Nobody's perfect
```

But a custom str is used if defined:

```
python >> class Oops(Exception):
```

```
def str(self): return'Custom display used'>> raise Oo  
ps("Nobody's perfect") ... Oops: Custom display used
```

See Chapter 30 for more details on these special operator-overloading methods.

中文翻译

如前一节所见，默认情况下，基于类的异常实例在捕获并打印时，会显示你传给类构造函数的任何内容：`python >>> class MyBad(Exception): pass >>> try: ... raise MyBad('Sorry--my mistake!') # 抛出带消息的异常... except MyBad as X: ... print(X) # 默认打印构造参数... Sorry--my mistake!` 当异常未被捕获、作为错误信息的一部分显示时，同样使用这种继承来的默认显示模型：`python >>> raise MyBad('Sorry--my mistake!') Traceback (most recent call last): File "<stdin>", line 1, in <module> MyBad: Sorry--my mistake!` 对很多角色来说，这已经足够了。但要提供更定制化的显示，你可以在类中定义两个字符串表示重载方法之一（`repr` 或 `str`），返回你希望异常显示的字符串。无论异常是被捕获并打印，还是到达默认处理器，这个方法返回的字符串都会显示出来：`python >>> class MyBad(Exception): def str(self): return'Stuff happens...' # 自定义显示：固定字符串>>> try: ... raise MyBad() ... except MyBad as X: ... print(X) ... Stuff happens... >>> raise MyBad() Traceback (most recent call last): File "<stdin>", line 1, in <module> MyBad: Stuff happens... 无`

论你的方法返回什么，都会包含在未捕获异常的错误信息中，并在异常被显式打印时使用。这里的返回是一个硬编码字符串，只是用来说明；它也可以做任意的文本处理，甚至可以使用挂在实例对象上的状态信息。下一节将讨论状态信息的选项。不过，先讲一个细微之处：出于异常显示的目的，你通常必须重定义 `str`，因为内置异常超类已经有一个 `str` 方法，而且在某些上下文中 `str` 优先于 `repr`——包括错误信息显示。如果你只定义 `repr`，打印时会愉快地调用内置超类的 `str` 而不是你的 `repr`：python >>> class Oops(Exception): def repr(self): return 'Custom display not used' # 只定义 repr 无效>>> raise Oops("Nobody's perfect")...Oops: Nobody's perfect 但如果你定义了 `str`，它就会被使用：python >>> class Oops(Exception): def str(self): return 'Custom display used' # 定义 str 才生效>>> raise Oops("Nobody's perfect")...Oops: Custom display used 关于这些特殊运算符重载方法的更多细节，参见第 30 章。

代码分析

```
>>> class MyBad(Exception):
...     def __str__(self):
...         return 'Stuff happens...' #
>>> try:
...     raise MyBad() # args
... except MyBad as X:
...     print(X) # str(X) -> MyBad.__str__
...
Stuff happens...

>>> class Oops(Exception):
...     def __repr__(self): return 'Custom display not used'
```

```
>>> raise Oops("Nobody's perfect")
Oops: Nobody's perfect                                # __repr__

>>> class Oops(Exception):
...     def __str__(self): return 'Custom display used'
>>> raise Oops("Nobody's perfect")
Oops: Custom display used                             # __str__
```

·解释器如何处理：print(X) 触发 str(X)，解释器查找 str 的解析顺序为：实例的类 (MyBad) → 沿 mro 向上。MyBad 自己定义了 str，直接命中，返回 'Stuff happens...'。未捕获异常生成错误信息时，traceback 处理器同样调用异常的 str 来拼类名：消息。而只定义 repr 的 Oops，由于 mro 上先找到 BaseException.str (print/错误信息上下文优先用 str)，你的 repr 根本不会被调用。

·设计思想：这条"必须重写 str"的提醒很有 Python 味道——显示语义 (str，给人看的消息) 和回声语义 (repr，机器可重构的表达式) 分工明确，错误信息走 str 通道。想改错误消息就改 str。

深度理解

·核心概念：自定义异常显示 = 重写 str。这是修改"未捕获错误信息"和"被捕获后 print 输出"的唯一正确入口。

·底层实现：BaseException 自带 str (返回 args 的显示) 和 repr (返回构造式文本)。子类重写 str 就覆盖了继承的实现 (Python 属性查找先命中子类)。traceback 模块构造错误最后一行时，对异常调用 str(exc) 得到消息文本。

·设计原因：为什么 print 优先用 str 而非 repr？这是 Python 的统一约定：str() 面向可读性，repr() 面向精确性；对异常来说错误信息显然属于“可读性”场景。作者特意用一个反例（只重写 repr 无效）把这个坑讲透。

·实际问题：让异常消息“人话化”——比如网络库的 TimeoutError 重写 str 返回“请求超时，请重试”，运维日志更友好；自定义消息还能动态拼入状态（文件名、行号等）

·初学者误区：① 重写了 repr 就以为显示会变——在错误信息和 print 场景中，str 优先；② 以为 str 必须返回 self.args[0] 之类的东西——其实可以返回任意字符串，甚至忽略 args；③ 想在 init 里重新组织消息文本，却在 str 里忘了解释 args——注意“显示”和“构造”是两条独立通道

Custom State and Behavior (自定义状态与行为)

英文原文

Besides supporting flexible hierarchies, exception classes also provide storage for extra state information as instance attributes. As discussed earlier, built-in exception superclasses provide a default constructor that automatically saves constructor arguments in an instance tuple attribute named args. Although the default constructor is adequate for many cases, for more

custom needs we can provide a constructor of our own. In addition, classes may define methods for use in handlers that provide precoded exception processing logic.

中文翻译

除了支持灵活的层次结构，异常类还可以用实例属性的形式为额外的状态信息提供存储。如前所述，内置异常超类提供了一个默认构造函数，自动把构造参数保存到名为 `args` 的实例元组属性中。虽然默认构造函数对许多情况都够用，但为了更定制化的需求，我们可以提供自己的构造函数。此外，类还可以定义供处理器使用的方法，提供预编码好的异常处理逻辑。

深度理解

- 核心概念：异常类 = 类（继承、方法）+ 实例（属性、状态）。`args` 是内置的通用状态，自定义属性是应用专属状态。
- 底层实现：重写 `init` 后，构造时先执行 `BaseException.init(args)`（你可以在自定义 `init` 里调用 `super().init()` 保留 `args`），再设置自己的属性。异常实例在 `except ... as X` 绑定后，属性、方法随使用。
- 设计原因：把状态放进异常实例而不是全局变量，是为了可靠地跨模块传播——`raise` 和 `try` 可能在不同文件，实例自带的属性跟着异常走，天然可靠。
- 实际问题：解析器、框架、库把错误细节（行号、文件名、错误码）直接塞进异常对象，处理器拿到的是“自解释”

的错误。

·初学者误区：① 重写 `init` 后忘了调用 `super().init()`，导致 `args` 为空、`str(exception)` 显示空——显示与状态双双失效；② 把状态存到类属性而非实例属性，导致所有异常实例共享同一份状态。

Providing Exception Details (提供异常细节)

英文原文

When an exception is raised, it may cross arbitrary file boundaries—the raise statement that triggers an exception and the try statement that catches it may be in completely different module files. It is not generally feasible to store extra details in global variables because the try statement might not know which file the globals reside in. Passing extra state information along in the exception itself allows the try statement to access it more reliably. With classes, this is nearly automatic. As we've seen, when an exception is raised, Python passes the class instance object along with the exception. Code in try statements can access the raised instance by listing an extra variable after the `as` keyword in an except handler. This provides a natural hook for supplying data and behavior to the handler. Generic instance-access tools like `sys.excinfo` used earlier enable the same interfaces. For example, a progr

am that parses data files might signal a formatting error by raising an exception instance that is filled out with extra details about the error (the input file here isn't real because this demo dies before reading it!):

```
python >>> class FormatError(Exception): def init(self, line, file): self.line = line self.file = file >>> def parser(file): # Parse the file first raise FormatError(62, file=file) # When an error is found >>> try: ... parser('code.py') ... except FormatError as X: ... print(f'Error at: {X.file} #{X.line}') # Custom state info ... Error at: code.py # 62
```

In the except clause here, the variable `X` is assigned a reference to the instance that was generated when the exception was raised. This gives access to the attributes attached to the instance by the custom constructor. Although we could rely on the default state retention of built-in superclasses, it's less relevant to our application (and doesn't support the keyword arguments used in the prior example):

```
python >>> class FormatError(Exception): pass # Inherited constructor >>> def parser(file): raise FormatError(file, 62) # No keywords allowed! >>> try: ... parser('code.py') ... except FormatError as X: ... print(f'Error at: {X.args[0]} #{X.args[1]}') # Generic state info ... Error at: code.py #62
```

中文翻译

异常被抛出时，可能穿越任意的文件边界——触发异常的 `raise` 语句和捕获它的 `try` 语句可能位于完全不同的模块文件中。把额外细节存到全局变量里通常不可行，因为 `try` 语句

可能不知道该全局变量存在于哪个文件。把额外的状态信息随着异常本身传递，能让 try 语句更可靠地访问它。有了类，这几乎是自动完成的。如我们所见，异常被抛出时，Python 会随异常一起传递类实例对象。try 语句中的代码通过在 except 处理器的 as 关键字后列出一个额外变量来访问被抛出的实例。这为向处理器提供数据和行为提供了一个自然的钩子。前面用到的 sys.excinfo 这类通用实例访问工具也能提供同样的接口。例如，一个解析数据文件的程序，可以通过抛出装满错误额外细节的异常实例来报告格式错误（这里的输入文件不是真实的，因为这个演示在读它之前就挂掉了！）：

```
python >>> class FormatError(Exception):
def init(self, line, file): # 自定义构造函数self.line = line
# 保存行号self.file = file # 保存文件名>>> def parser(file): # 先解析文件raise FormatError(62, file=file) # 发现
错误时抛出>>> try: ... parser('code.py') ... except Form
atError as X: ... print(f'Error at: {X.file} #{X.line}') # 自定义
状态信息... Error at: code.py #62 这里的 except 子句
中，变量 X 被赋为异常抛出时生成的实例的引用。这让我们
能访问自定义构造函数挂到实例上的属性。虽然我们可以
依赖内置超类的默认状态保留，但它与我们的应用关系不大
（而且不支持前一示例中使用的关键字参数）：python >
>> class FormatError(Exception): pass # 继承的构造函数>>> def parser(file): raise FormatError(file, 62) # 关
键字参数不行！>>> try: ... parser('code.py') ... except F
ormatError as X: ... print(f'Error at: {X.args[0]} #{X.args
[1]}') # 通用状态信息... Error at: code.py #62
```

代码分析

```
>>> class FormatError(Exception):
...     def __init__(self, line, file):    #
...         self.line = line             #
...         self.file = file             #
>>> def parser(file):                    #
...     raise FormatError(62, file=file) #
>>> try:
...     parser('code.py')
... except FormatError as X:              # X
...     print(f'Error at: {X.file} #{X.line}') #
...
Error at: code.py #62
```

·解释器如何处理：FormatError(62, file=file) 构造时执行自定义 init：line 参数位置传 62，file 用关键字传'code.py'，分别挂到实例的 line、file 属性。raise 把这个实例传播出去；except FormatError as X 匹配成功后，把实例引用赋给 X。处理器里 X.line、X.file 就是属性访问，毫无魔法。注意：自定义 init 没调用 super().init()，所以 args 是空的——这不是 bug，因为你不需要它了。

·对照：用继承的默认构造函数时，FormatError(file, 62) 只接受位置参数（默认 init(args) 收集一切，但不接受关键字参数——因为 args 收集位置参数，关键字会 TypeError），访问方式退化为 X.args[0]、X.args[1]，可读性差。

·设计思想：自定义 init 的价值 = 语义化属性名 + 关键字参数支持。X.file、X.line 比 X.args[0]、X.args[1] 自解释得多——这正是作者说的“更贴近应用”。

深度理解

·核心概念：异常携带状态的两种模式——①默认模式：args 元组，零配置；②自定义模式：重写 init 存语义化属性。按需选择。

·底层实现：异常实例就是普通对象，self.line = line 就是普通属性赋值。特殊之处只在传播路径：raise → 栈展开 → except 绑定，实例的引用被原样传递，属性自然跟着走。

·设计原因：为什么不用全局变量传状态？因为 raise 和 try 可能在不同模块文件，"try 端"无法可靠地知道状态存在哪个模块的 globals 里；而异常实例自带状态，随异常走，天然解耦。这是"异常即对象"模型最有说服力的工程理由。

·实际问题：这是所有正经库的标准做法——OSError 的 filename 属性、requests 的 response 属性、Django 的 Http404 自定义属性，都是这个模式。解析器、编译器、框架的错误对象几乎都这么做。

·初学者误区：① 自定义 init 后忘调 super().init(), 然后发现 str(exc) 空、args 空——如果想保留"传参即显示"的默认能力，就要调 super().init(line, file); ② 试图在 except 里通过局部变量/全局变量拿到错误上下文——别绕路，实例属性才是正道；③ 自定义 init 时参数名与内置 args 冲突之类的命名混乱。

Providing Exception Methods (提供异常方法)

英文原文

Besides enabling application-specific state and display, classes also support extra behavior for exception objects. That is, the exception class can also define unique methods to be called in handlers. The file `parsely.py` in Example 35-3, for example, adds a method that uses exception custom state information to log errors to a file automatically.

Example 35-3. `parsely.py`

```
python from time import asctime class FormatError(Exception): logfile = 'parser-errors.txt' def init(self, line, file): self.line = line self.file = file def logerror(self): with open(self.logfile, 'a') as log: print(f'Error at: {self.file} #{self.line} [{asctime()}]', file=log) def parser(file): # Parse a file here... raise FormatError(line=62, file=file) if name == 'main': try: parser('code.py') except FormatError as exc: exc.logerror()
```

When run, this script appends its error message to a file in response to method calls in the exception handler (use `type` instead of the Unix `cat` on Windows, and see Python manuals for `time.asctime`):

```
$ python3 parsely.py $ python3 parsely.py $ cat pars
```

er-errors.txt Error at: code.py #62 [Sat Jul 13 12:22:19 2024] Error at: code.py #62 [Sat Jul 13 12:22:25 2024]

In such a class, methods (like `logerror`) may also be inherited from superclasses, and instance attributes (like `line` and `file`) provide a place to save state information that provides extra context for use in later method calls.

Moreover, exception classes are free to customize and extend inherited behavior:

```
python class CustomFormatError(FormatError): def logerror(self):... something unique here .....
```

```
raise CustomFormatError(...)...
```

```
try:...
```

```
except FormatError as exc: exc.logError() # Runs the raised class's version
```

In other words, because they are defined with classes, all the benefits of OOP that we studied in Part VI are available for use with exceptions in Python.

Two final notes here: first, the raised instance object assigned to `exc` in this code is also available generically as the second item in the result tuple of the `sys.exc_info()` call used earlier—a tool that returns information about the exception being handled. This call can be used if you do not list an exception name in an `except` clause but still need access to the exception that occurred, or to any of its attached state information or

methods. And second, although our class's `logerror` method appends a custom message to a logfile, it could also generate Python's standard error message with stack trace using tools in the `traceback` standard-library module, which uses `traceback` objects.

To learn more about `sys.excinfo` and `tracebacks`, though, we need to move ahead to the next chapter.

中文翻译

除了支持应用专属的状态和显示，类还为异常对象支持额外行为。也就是说，异常类还可以定义在处理器中被调用的独特方法。例如，示例 35-3 中的文件 `parsely.py` 添加了一个方法，利用异常的自定义状态信息把错误自动记录到文件里。示例 35-3. `parsely.py`

```
python from time import asctime
class FormatError(Exception):
    logfile = 'parser-errors.txt'
    def init(self, line, file):
        self.line = line
        self.file = file
    def logerror(self):
        with open(self.logfile, 'a') as log:
            print(f'Error at: {self.file} #{self.line} [{asctime()}]', file=log)
    def parser(file):
        # Parse a file here...
        raise FormatError(line=62, file=file)
if name == 'main':
    try:
        parser('code.py')
    except FormatError as exc:
        exc.logerror()
```

运行时，这个脚本会根据异常处理器中的方法调用，把错误消息追加到文件中（Windows 上用 `type` 而不是 Unix 的 `cat`；关于 `time.asctime` 见 Python 手册）：

```
$ python3 parsely.py
$ python3 parsely.py
$ cat parser-errors.txt
Error at: code.py #62 [Sat Jul 13 12:22:19 2024]
Error at: code.py #62 [Sat Jul 13 12:22:25 2024]
```

在这样的类中，方法（如 `logerror`）也可以从超类继承，实例属性

(如 line 和 file) 提供了保存状态信息的位置，为后续的方法调用提供额外的上下文。此外，异常类可以自由地定制和扩展继承来的行为：python class CustomFormatError (FormatError): def logerror(self): # 重写方法，做自己的处理...这里做点独特的处理.....raise CustomFormatError(...) # 抛出子类实例...try:...except FormatError as exc: # 捕获超类exc.logError() # 运行的是被抛出类的版本（多态！）换句话说，正因为异常是用类定义的，我们在第 VI 部分学到的 OOP 的全部好处都可以用在 Python 的异常上。最后两点说明：第一，这段代码中赋给 exc 的被抛出实例，也可以通用地通过前面用过的 sys.excinfo() 调用的结果元组的第二项获得——这个工具返回当前正在处理的异常的信息。当你在 except 子句中没有列出异常名、却仍需要访问已发生的异常（或它的任何状态信息、方法）时，可以使用这个调用。第二，虽然我们类的 logerror 方法只是把自定义消息追加到日志文件，它也可以用标准库 traceback 模块中的工具（基于 traceback 对象）生成带堆栈跟踪的 Python 标准错误消息。不过，要了解更多关于 sys.excinfo 和 traceback 的内容，我们需要进入下一章。

代码分析

```
from time import asctime                                #

class FormatError(Exception):
    logfile = 'parser-errors.txt'                        #

    def __init__(self, line, file):                      #
        self.line = line
        self.file = file
```

```

def logerror(self):
    #
    with open(self.logfile, 'a') as log: #
        print(f'Error at: {self.file} #{self.line} [{asctime()}...
            #

def parser(file):
    # ...
    raise FormatError(line=62, file=file) #

if __name__ == '__main__':
    #
    try:
        parser('code.py')
    except FormatError as exc:
        exc.logerror() #

```

·解释器如何处理：parser('code.py') 抛出 FormatError 实例（带着 line=62、file='code.py'）。except FormatError as exc 匹配并把实例绑到 exc。exc.logerror() 是普通的方法调用——logerror 从类里取到，self 绑定到被抛出的实例，因此方法内能读 self.file、self.line，以追加模式（'a'）打开类属性 logfile 指定的文件，把格式化的错误消息写入。运行两次后，日志文件里就有两条记录（asctime() 给出不同的时间戳）。if name == 'main': 保证它被 import 时不执行演示代码。

·多态要点：except FormatError as exc: exc.logerror() 中，如果实际抛出的是子类 CustomFormatError 的实例，方法调用会沿 mro 找到子类重写后的 logerror——同一个 except 子句，自动执行对应子类的版本。这正是本书一直在讲的 OOP 多态（polymorphism）在异常世界的应用：处理器不关心具体异常类，只调用接口，行为自动分派。

·设计思想：把"怎么处理这个错误"编进异常类的方法里，处理器就变得极其简单——`except FormatError as exc: exc.logerror()` 一行搞定。这也是"异常即对象"思想的巅峰体现：异常不只是信号，还是带行为的对象。

深度理解

·核心概念：异常类可以定义方法，处理器直接调用。异常从"信号"升级为"带智能的对象"——错误怎么记录、怎么恢复，都可以封装在异常类里。

·底层实现：方法调用就是普通的对象方法分派（沿 `mro` 查找），配合 `as` 绑定的实例，`self` 指向被抛出的异常实例。没有任何特殊机制——这正是 OOP 的力量。

·设计原因：为什么把处理逻辑放进异常类而不是处理器里？① 处理逻辑随异常走，`try` 端代码极简；② 子类可重写处理逻辑，实现多态处理；③ 状态（`line/file`）+ 行为（`logerror`）内聚在同一个对象里。

·实际问题：`sys.excinfo()` 的元组第二项与 `as` 绑定的实例是同一个对象；在裸 `except:` 里没有 `as`，仍可 `sys.excinfo()[1].logerror()`。`traceback` 标准库可以生成带堆栈的错误文本，是生产级日志的标准做法。

·初学者误区：① 在 `except` 里重复写一堆处理代码，而不是把处理逻辑封装到异常类；② 忘记异常方法能重写、能继承——以为异常类只能有 `pass`；③ 混淆 `sys.excinfo()[0]`（类）和 `sys.excinfo()[1]`（实例），实例才带属性方法。

Exception Groups: Yet Another Star! (异常组：又一颗新星！)

英文原文

But wait—just when you thought it was safe to put exceptions in the win column, the exceptions story has recently sprouted yet another wild plot twist, which is sufficiently limited and arcane to pass as an optional follow-up topic for most Python learners (and was deferred until now for this reason). Lest it crop up in one of those silly job interviews that favor the inane over the practical, though, here's a quick peek.

Let's get right to the code. As we've seen, try statements normally run at most one matching handler clause, plus an optional finally on exit:

```
python >> try:
```

```
... raise IndexError() ... except IndexError: ... print('Got IE') ... except (SyntaxError, TypeError): ... print('Got SE') ... Got IE
```

Python 3.11, though, adds the ability to trigger multiple matching handlers in a single try statement by wrapping them in an exception group and catching them with except clauses:

```
python >> try:
```



```
... raise ExceptionGroup('Many', [IndexError(), SyntaxError()]) ... except IndexError: ... print('Got IE') ... except (SyntaxError, TypeError): ... print('Got SE') ... Got IE Got SE
```

In a nutshell, each `except` clause can process and consume one exception, or one batch of them, in the group. Here, the first clause runs for `IndexError` and the second for `SyntaxError` (a tuple means "any" as before). The group is simply an exception object made by calling a built-in exception class provided for this role, passing a message-string label used in displays, along with a sequence of exceptions to be matched by `except` clauses in a `try`.

Syntactically, an empty `except` is not allowed, and a basic `except` cannot be mixed with `except`—but an `else` and `finally` can. Moreover, `except` cannot host a `break`, `continue`, or `return`—but `except` can. Like the awkward `except/else/finally` mixing rules before it, these special cases probably qualify `except` as a distinct statement form; adding it to the mix makes `try` an overloaded jumble of semi-related functionality.

Semantically, each `except` clause executes at most once, and consumes all matching exceptions in the group. In addition, each exception in the group is handled by at most one `except` clause—the topmost clause that matches it; optional as variables are assigned exception groups—with attributes like exceptions that

expose their contents; and any unmatched exceptions in the group are reraised after the try statement processes matches—with a top-level message that denotes those unmatched:

```
python >> excs = ExceptionGroup('Many', [IndexError(), SyntaxError(), TypeError()]) >> try:
```

```
... raise excs ... except IndexError as E: ... print(f'Got IE: {E} => {E.exceptions}') ... except SyntaxError as E: ... print(f'Got SE: {E} => {E.exceptions}') ... Got IE: Many (1 sub-exception) => (IndexError(),) Got SE: Many (1 sub-exception) => (SyntaxError(),) + Exception Group Traceback (most recent call last): | File "<stdin>", line 2, in <module> | ExceptionGroup: Many (1 sub-exception) +-+----- 1 ----- | TypeError +-----
```

When the group has multiple exceptions of the same type, a matching except consumes them all and can process them in normal iteration code. As also shown next, spaces around the are allowed and ignored—despite all the except labels in docs, and more reflective of the wildcard nature of these clauses:

```
python >> try:
```

```
... raise ExceptionGroup('Dups', [IndexError(), TypeError(1), TypeError(2)]) ... except IndexError: ... print('Got IE') ... except TypeError as E: ... print(f'Got TE: {E} => {E.exceptions}') ... Got IE Got TE: Dups (2 sub-exceptions) => (TypeError(1), TypeError(2))
```

As usual, a group's exception matches an except that names its class or one of its superclasses. Because of this, the ordering of clauses is both subtle and important—the first match wins and removes exceptions from the group. In the following, for example, the first clause gobbles `IndexError` via its `LookupError` superclass, along with the two `TypeError`s in the group (but reversing the clauses' order would handle `IndexError` separately):

```
python >> try:
```

```
... raise ExceptionGroup('Dups', [IndexError(), TypeError(1), TypeError(2)]) ... except (TypeError, LookupError) as E: ... print('Got1:', E) ... except IndexError as E: ... print('Got2:', E) ... Got1: Dups (3 sub-exceptions)
```

The except can also match basic individual exceptions, which are automatically wrapped in a group to appease group-based code in the handler:

```
python >> try:
```

```
... raise IndexError ... except IndexError as E: ... print(f'Got IE: {E} => {E.exceptions}') ... Got IE: (1 sub-exception) => (IndexError(),)
```

And a basic except can catch a group as a collective and process it manually, but an except cannot catch a group because it would be ambiguous (a schism of the sort that's usually a hallmark of an ad hoc extension):

```
python >> try:
```

```
... raise ExceptionGroup('Lots', [IndexError(), SyntaxError()]) ... except Exception as E: ... print(f'Got group: {E} => {E.exceptions}') ... for exc in E.exceptions: ... print('With exc:', type(exc)) ... Got group: Lots (2 sub-exceptions) => (IndexError(), SyntaxError()) With exc: <class'IndexError'> With exc: <class'SyntaxError'>
```

Finally, for comparison, here's a catchall in both models—though there's no reason to use `except` to catch a single exception (unless overly complicated code is your thing):

```
python >> try:
```

```
... raise ExceptionGroup('Dups', [IndexError()]) ... except Exception as E: ... print(type(E.exceptions[0])) ... <class'IndexError'> >> try:
```

```
... raise IndexError() ... except Exception as E: ... print(type(E)) ... <class'IndexError'>
```

For another exception-groups example, see the next chapter's Example 36-2, which demos how runtime nesting consumes items in groups (short story: groups propagate until empty, then die like individual exceptions).

Design concerns aside, the "why" of `except` is even more elusive than the "how." While it's conceivable that some programs may wish to collect a set of exceptions and send them to a `try` statement as a batch, it's harder to understand why these wildly rare programs

could not be expected to package with normal exception objects and process with normal iteration code—instead of convoluting the Python language for everyone.

Because exception groups are an obscure tool with very narrow roles, we'll defer to Python's manuals for more info on this esoteric try extension that, like many a Python mod, seems to blow up complexity radically to address a purported need that somehow managed to go unnoticed for all of Python's first three decades+. How did we live?

中文翻译

但是等等——就在你以为异常这部分可以记在“赢”这一栏的时候，异常的故事最近又冒出了一个狂野的情节转折。它的适用范围足够狭窄、足够冷门，对大多数 Python 学习者来说可以作为可选的后续话题（也正因如此被推迟到现在才讲）。不过，为了防止它出现在那些偏爱无聊问题而非实用问题的蠢面试题里，这里还是快速瞄一眼吧。直接看代码。如我们所见，try 语句通常最多运行一个匹配的处理子句，外加退出时可选的一个 finally：python >>> try: ... raise IndexError() ... except IndexError: ... print('Got IE') ... except (SyntaxError, TypeError): ... print('Got SE') ... Got I E 不过，Python 3.11 增加了在单个 try 语句中触发多个匹配处理器的能力：把多个异常包进一个异常组（exception group），再用 except 子句捕获它们：python >>> try: ... raise ExceptionGroup('Many', [IndexError(), SyntaxError()]) ... except IndexError: ... print('Got IE') ... except

(SyntaxError, TypeError): ... print('Got SE') ... Got IE Got SE 简而言之，每个 except 子句可以处理并消耗组中的一个异常，或一批异常。这里第一个子句为 IndexError 运行，第二个为 SyntaxError 运行（元组如前表示“任一”）。组（group）就是一个异常对象，通过调用为此角色提供的内置异常类来创建，传入一个用于显示的消息字符串标签，以及一个供 try 中 except 子句匹配的异常序列。语法上，不允许空 except，基本 except 不能与 except 混用——但 else 和 finally 可以。此外，except 中不能出现 break、continue 或 return——而 except 可以。就像之前 except /else/finally 那别扭的混用规则一样，这些特殊情况可能足以让 except 成为一种独立（distinct）的语句形式；把它加进这个组合，让 try 变成了一锅半相关功能的超载大杂烩。语义上，每个 except 子句最多执行一次，并消耗组中所有匹配的异常。此外，组中的每个异常至多被一个 except 子句处理——即最顶层的匹配子句；可选的 as 变量被赋为异常组——带有 exceptions 等属性来暴露其内容；组中任何未匹配的异常，在 try 语句处理完匹配之后会被重新抛出——带一条顶层消息表明哪些未匹配：python >>> excs = ExceptionGroup('Many', [IndexError(), SyntaxError(), TypeError()]) >>> try: ... raise excs ... except IndexError as E: ... print(f'Got IE: {E} => {E.exceptions}') ... except SyntaxError as E: ... print(f'Got SE: {E} => {E.exceptions}') ... Got IE: Many (1 sub-exception) => (IndexError(),) Got SE: Many (1 sub-exception) => (SyntaxError(),) + Exception Group Traceback (most recent call last): | File "<stdin>", line 2, in <module> | ExceptionGroup: Many (1 sub-exception) +-+----- 1 ---

-----| TypeError +-----

-----当组中有多个同类型异常时，匹配的 except 会全部消耗掉，并可以在普通的迭代代码中处理它们。如下所示，周围允许并有忽略的空格——尽管文档里全是 except 标签，这更体现了这些子句的通配符 (wildcard) 性质：python

```
>>> try: ... raise ExceptionGroup('Dups', [IndexError(),
, TypeError(1), TypeError(2)]) ... except IndexError: ... p
rint('Got IE') ... except TypeError as E: ... print(f'Got TE:
{E} => {E.exceptions}') ... Got IE Got TE: Dups (2 sub-e
xceptions) => (TypeError(1), TypeError(2)) 像往常一样
，组中的异常匹配写有其类名或其超类名的 except。正因
如此，子句的顺序既微妙又重要——最先匹配的子句获胜
并把异常从组中移除。例如下面，第一个子句通过 IndexEr
ror 的 LookupError 超类把 IndexError 连同组里的两个 T
ypeError 一起吞掉（但如果颠倒子句顺序，IndexError 会
被单独处理）：python >>> try: ... raise ExceptionGrou
p('Dups', [IndexError(), TypeError(1), TypeError(2)]) ...
except (TypeError, LookupError) as E: ... print('Got1:',
E) ... except IndexError as E: ... print('Got2:', E) ... Got1:
Dups (3 sub-exceptions) except 也能匹配基本的单个异
常，单个异常会被自动包装进一个组，以迎合处理器中基于
组的代码：python >>> try: ... raise IndexError ... excep
t IndexError as E: ... print(f'Got IE: {E} => {E.exceptions
}') ... Got IE: (1 sub-exception) => (IndexError(),) 而且
基本 except 可以整体捕获一个组并手动处理它，但 excep
t 不能捕获一个组，因为那会产生歧义（这种分裂通常是一种
ad hoc（临时拼凑）扩展的标志）：python >>> try: .
.. raise ExceptionGroup('Lots', [IndexError(), SyntaxErr
```

or())) ... except Exception as E: ... print(f'Got group: {E} => {E.exceptions}') ... for exc in E.exceptions: ... print('With exc:', type(exc)) ... Got group: Lots (2 sub-exceptions) => (IndexError(), SyntaxError()) With exc: <class'IndexError'> With exc: <class'SyntaxError'> 最后，做个对比，下面是两种模型下的"万能捕获"——不过没有理由用 except 去捕获单个异常（除非你就喜欢过度复杂的代码）：

```
python >>> try: ... raise ExceptionGroup('Dups', [IndexError()]) ... except Exception as E: ... print(type(E.exceptions[0])) ... <class'IndexError'> >>> try: ... raise IndexError() ... except Exception as E: ... print(type(E)) ... <class'IndexError'>
```

另一个异常组示例见下一章的示例 36-2，它演示了运行时嵌套如何消耗组中的条目（简言之：组一直传播直到变空，然后像单个异常一样消亡）。抛开设计上的担忧不谈，except 的"为什么"比"怎么做"更加难以捉摸。虽然可以想象某些程序希望收集一组异常，作为一个批次发送给 try 语句，但很难理解为什么这些极其罕见的程序不能指望用普通异常对象打包、用普通迭代代码处理——反而要为所有人把 Python 语言搞得复杂化。因为异常组是一个用途极窄的冷门工具，关于这个深奥的 try 扩展的更多信息，我们请读者查阅 Python 手册。它就像很多 Python 的修改一样，似乎是为了一个"Python 头三十年+都安然无恙地没被发现的需求"而急剧增加复杂性。我们以前是怎么活过来的？

代码分析


```
>>> try:
...     raise ExceptionGroup('Many', [IndexError(), SyntaxError()])...
... except* IndexError:                # IndexError
...     print('Got IE')
... except* (SyntaxError, TypeError): # "" SyntaxErro...
...     print('Got SE')
...
Got IE
Got SE                                # try
```

·解释器如何处理：ExceptionGroup('Many', [...]) 是 Python 3.11+ 的内置类，构造时第一个参数是显示标签字符串，第二个是异常列表。raise 抛出这个组对象后，解释器把组内每个异常与 except 子句逐个匹配（同样的 instance 规则），每个子句“消耗”其匹配的所有异常并执行一次。所以上面的 try 打印了两行——这在普通 except 里不可能发生（普通 except 最多执行一个匹配子句）。

·关键语义：① 每个 except 最多执行一次；② 组中每个异常至多被一个子句处理——最先匹配的；③ as 绑定的是组（不是单个异常），用 E.exceptions 访问组内剩下的异常；④ 未匹配的异常在 try 结束后重新抛出（上面的 TypeError 就是这样，traceback 显示为 + Exception Group Traceback 格式）；⑤ 单个普通异常被 except 捕获时自动包装成组；⑥ except 能整体接住组（把组当一个异常处理），except 不能捕获“组中套组”这种歧义情形。

·设计思想：异常组来自 PEP 654，初衷是支持 asyncio 中并发任务各自抛出异常、需要批量汇报的场景。作者态度鲜明：这是小众工具，“为了罕见需求把 try 搞得复杂”

。对你的实际意义是：听说过、见过即可，日常工作几乎用不到。

深度理解

·核心概念：异常组 (ExceptionGroup) + except 子句
允许一个 try 运行多个处理器。组是对象，有 exceptions 属性；except 是配套的"批量消费"语法。

·底层实现：Python 3.11 引入 (PEP 654) ， Exception Group 是 BaseException 的子类，内部持有一个异常列表。except 匹配时对组内元素逐个做 isinstance 检查，命中的全部移出组；剩余的未匹配异常在 try 结束后组成新的组继续传播（这就是 traceback 里 + Exception Group Traceback 树状显示的来源）。except 与 except 不能混用、except 里不能 break/continue/return 等语法限制，都是实现层面的约束。

·设计原因：动机来自并发编程——asyncio.gather() 批量跑任务，多个任务同时失败时，传统模型只能抛一个异常，其他失败信息全丢；异常组让所有失败能被一并汇报。作者吐槽的正是：这个需求极其罕见，却换来一套复杂的语法。

·实际问题：asyncio 任务组 (TaskGroup) 、Exception Group 在并发代码里偶尔出现；99% 的日常业务代码用不上。作者的建议很实用：读文档了解即可，别主动用。

·初学者误区：① 把 except 当 except 用——except IndexError 绑定的 E 是组不是异常，要取 E.exceptions[0]；② 不知道未匹配的异常会被重新抛出——看到 traceba

ck 别以为 try 处理失败了；③ 在 3.11 之前的 Python 里用 ExceptionGroup 直接 NameError——这是新版本特性。

Chapter Summary (本章小结)

英文原文

In this chapter, we explored both built-in exceptions and ways to code exceptions of our own. As we learned, exceptions are implemented as class instance objects. Exception classes support the concept of exception hierarchies that ease maintenance, allow data and behavior to be attached to exceptions as instance attributes and methods, and allow exceptions to inherit tools from superclasses as usual in OOP. We saw that in a try statement, catching a superclass catches that class as well as all subclasses below it in the class tree—superclasses become exception category names, and subclasses become more specific exception types within those categories. We also saw that the built-in exception superclasses we must inherit from provide usable defaults for printing and state retention, which we can override if desired. Finally, armed with our new knowledge of exception objects, we also peeked at exception groups and the except clause, used to run multiple handlers in a try. We questioned whether this extension's convolution of try statements is j

ustified by its perceived roles; it's a lot to ask of most Python users, but this question is ultimately yours to answer. The next chapter wraps up this part of the book by exploring some common use cases for exceptions and surveying tools commonly used by Python programmers. Before we get there, though, here's this chapter's quiz.

中文翻译

在本章中，我们探讨了内置异常以及编写自定义异常的方法。如我们所学，异常被实现为类实例对象。异常类支持异常层次结构的概念——这减轻了维护负担，允许把数据和行为以实例属性和方法的形式附加到异常上，并允许异常像 OOP 中一贯的那样从超类继承工具。我们看到：在 try 语句中，捕获一个超类不仅捕获该类本身，还捕获类树中其下的所有子类——超类成为异常类别名，子类成为这些类别中更具体的异常类型。我们还看到，我们必须继承的内置异常超类提供了可用的打印和状态保留默认实现，如果需要，我们可以覆盖它们。最后，带着异常对象的新知识，我们还瞥了一眼异常组和 except 子句——用于在 try 中运行多个处理器。我们质疑了这种对 try 语句的复杂化扩展，是否配得上它所谓的用途；这对大多数 Python 用户要求过高，但这个问题最终由你自己回答。下一章将探索异常的一些常见用例、盘点 Python 程序员常用的工具，为本书这一部分画上句号。在进入下一章之前，先做一下本章的测验吧。

Test Your Knowledge: Quiz (自测: 小测验)

英文原文

1. What are the two main constraints on user-defined exceptions in Python? 2. How are raised exceptions matched to except handler clauses? 3. Name two ways that you can attach context information to exception objects. 4. Name two ways that you can specify the error-message text for exception objects. 5. What do except clauses do in a try statement?

中文翻译

1. Python 中用户自定义异常的两个主要约束是什么? 2. 被抛出的异常是如何匹配到 except 处理器子句的? 3. 说出两种把上下文信息附加到异常对象的方法。4. 说出两种为异常对象指定错误消息文本的方法。5. except 子句在 try 语句中做什么?

Test Your Knowledge: Answers (自测: 答案)

英文原文

1. Exceptions must be defined by classes (that is, a class instance object is raised and caught). In addition,

exception classes must be derived from the built-in class `BaseException` or one of its subclasses; most programs inherit from its `Exception` subclass to support catchall handlers for normal kinds of exceptions.

2. Exceptions match by superclass relationships: naming a superclass in an exception handler will catch instances of that class, as well as instances of any of its subclasses lower in the class tree. Because of this, you can think of superclasses as general exception categories and subclasses as more specific types of exceptions within those categories.

3. You can attach context information to exceptions with either custom or default constructors. A custom constructor can fill out attributes in a raised instance object that are specific to the program. For simpler needs, built-in exception superclasses provide a default constructor that stores its arguments on the instance automatically as tuple attribute `args`. Handlers can list a variable to be assigned to the raised instance, then go through this name to access attached state information and call any methods defined in the class.

4. The error-message text in exceptions can be specified with a custom `str` operator-overloading method. For simpler needs, built-in exception superclasses automatically display anything you pass to the class constructor. Operations like `print` and `str` automatically fetch the display string of an exception object when it is printed either explicitly or as part of an error message.

5. In a `try`, `except` is used to

o run possibly multiple handlers for exceptions raised as part of a group. The `except` also comes with heavy semantics, has special-case syntax and rules, does not combine with basic `except`, and is rarely useful in most Python programs. Nevertheless, there it is.

中文翻译

1. 异常必须用类来定义（也就是说，抛出和捕获的是类实例对象）。此外，异常类必须派生自内置类 `BaseException` 或其某个子类；大多数程序继承它的 `Exception` 子类，以支持对普通异常类型的“万能捕获”处理器。2. 异常通过超类关系匹配：在异常处理器中写一个超类，会捕获该类的实例，以及类树中位于其下的任何子类的实例。正因如此，你可以把超类看作通用的异常类别，把子类看作这些类别中更具体的异常类型。3. 你可以用自定义构造函数或默认构造函数把上下文信息附加到异常上。自定义构造函数可以在被抛出的实例对象中填上程序专属的属性。对于更简单的需求，内置异常超类提供默认构造函数，把其参数自动以元组属性 `args` 的形式存在实例上。处理器可以列出一个变量来接收被抛出的实例，然后通过这个名字访问附加的状态信息，并调用类中定义的任何方法。4. 异常的错误消息文本可以用自定义的 `str` 运算符重载方法指定。对于更简单的需求，内置异常超类会自动显示你传给类构造函数的任何内容。像 `print` 和 `str` 这样的操作会自动获取异常对象的显示字符串——无论它是被显式打印，还是作为错误消息的一部分打印。5. 在 `try` 中，`except` 用于为作为组的一部分抛出的异常运行可能的多个处理器。`except` 还带着沉重的语义，有特例语法和规则，不能与基本 `except` 混用，而且在大多数

Python 程序中很少有用武之地。不过，它就在那里。

本章总结

技术拓展 (Technical Expansion)

实际项目中的应用场景

- 自定义异常体系：任何一个"正经"的库都应该提供库名Error(Exception) 超类 + 若干子类，如 requests.RequestException → Timeout、ConnectionError 等；调用方 except requests.RequestException 一行覆盖全部。
- 异常携带上下文：Web 框架、ORM、解析器把参数校验失败、行号、字段名放进异常属性，上层直接读取渲染错误页面或记录日志。
- 错误日志：异常类自带 logerror 之类的方法（如本章 parse.py），或用 logging 模块 + excinfo=True 记录带 traceback 的完整日志。
- 异常组（3.11+）：asyncio.TaskGroup 批量任务的失败汇报、ExceptionGroup 手动合并多个异常时偶尔用到；日常 CRUD 项目基本不需要。

与其他语言 (Java/C++) 的区别

维度	Python	Java	C++
异常本质	类实例对象，任何类都可作为异常	必须继承 Throwable (或 Exception/Runtime Exception)	任何类型都能 throw (通常是类或基础类型)

声明	无需在函数签名里声明会抛什么异常（鸭子类型）	检查型异常必须 throws 声明	用 noexcept 可选声明
自定义惯例	继承 Exception（继承 BaseException 被禁止/不推荐）	继承 Exception 或其子类；检查型 vs 运行时	随意，惯例上继承 std::exception
匹配	按继承树 isinstance 匹配，except Exception 万能兜底	按 catch 类型自上而下匹配	按 catch 类型匹配
状态	args 元组 + 自定义属性 + 方	常用 message + cause 链	what() 方法
是否强制处理	不强制（裸 raise 传播，程序崩溃）	检查型异常强制处理，否则编译不过	不强制，崩溃由 terminate 兜底

Python 发展历史背景

- 1989–1995：Python 1.x 时代异常可以是字符串——raise 'error occurred'。字符串异常没有状态、不能继承，except 'error' 只能精确匹配，维护困难。
- 1998：Python 1.5 引入类异常，正式确立“异常是类实例对象”的模型；字符串异常在 Python 2.0（2000）被官方弃用，2.5 之后移除。这是本章开头“the prior chapter's note lamented”所指的历史包袱。
- 1998：同时确立“用户异常必须继承 BaseException”的约束——为什么？因为解释器需要保证 except Exception 兜底的可靠性，并让 KeyboardInterrupt 等系统事件独立于普通错误。
- 2022：Python 3.11 按 PEP 654 引入 ExceptionGroup 与 except，服务于并发编程的批量异常场景——作者在书中对此的吐槽，与社区“99% 用不上”的普遍判断一致

。

高级开发者需要掌握的相关知识

- `BaseException` 的完整继承树 (`SystemExit`、`KeyboardInterrupt`、`GeneratorExit` 为什么不在 `Exception` 下)

。

- `except` 与 `as` 的绑定生命周期: `as` 变量在 `except` 块结束后被删除 (Python 3 引入), 避免 `traceback` 循环引用。

- `raise ... from ...` (异常链) 与 `cause`、`context` 属性——下一章会展开。

- `traceback` 对象与 `traceback` 模块、`sys.excinfo()`、`sys.exception()` (3.11+)。

- 在自定义异常中调用 `super().init(args)` 保留 `args` 的策略; `BaseExceptionGroup` 与 `ExceptionGroup` 的区别 (3.11+)。

学习建议 (Learning Advice)

- 重要程度: ★★★★★ (4/5)。异常处理是 Python 日常开发的核心技能, 本章是"异常机制的原理课", 比上一章(语法)更需要理解深度。

- 应该掌握到什么程度:

- 必须熟练: 自定义异常三步走 (继承 `Exception`、`pass` 或自定义体、`raise` 实例); `except ... as` 绑定实例; `args` 与 `str` 的默认/自定义行为; 按继承树匹配。

·应该理解：raise X vs raise X() 的等价性及实例概念；为什么 except Exception 是推荐兜底而裸 except: 不是；sys.excinfo() 三要素（类、实例、traceback）。

·了解即可：ExceptionGroup/except（记住"3.11+、极少用"即可，面试可能问）。

·后续学习：下一章（第 36 章）是异常处理的实战收尾——try/except/else/finally 的组合语义、raise from 异常链、sys.excinfo 与 traceback 工具、异常处理的常见陷阱（gotcha）；随后进入类的第三部分（第 37 章起）时，可以反过来用本章的"异常即对象"理解装饰器、描述符等更多"对象化"设计。

·动手练习建议：① 用 isinstance 验证 ZeroDivisionError 是否是 ArithmeticError 的子类；② 给自己的练习库定义"类别超类 + 三个子类"，写一个 except 类别的处理器跑通；③ 自定义一个带 init（保存行号/文件名）和 str（返回友好消息）的异常，在 try 里捕获后打印。